

REPORT

Zajęcia: Analog and digital electronic circuits

Teacher: prof. dr hab. Vasyl Martsenyuk

Lab 8

Date 24.04.26

Topic: "Application of Digital Signal Processing in
Biomedical Signal Processing: ECG, EEG, EMG
Analysis"

Variant 13

Kostiantyn Hrekov
Informatyka II stopień,
stacjonarne,
1 semestr,
Gr 1.a

1. Problem statement:

Variant 13.

Each student is assigned a variant of the same ECG processing task. The goal is to:

- Preprocess a simulated or real ECG signal.
- Apply bandpass filtering to remove baseline drift and high-frequency noise.
- Detect R-peaks in the filtered signal.
- Calculate and plot heart rate over time.
- Interpret the results based on signal quality and parameter choices.

The general structure of the task:

1. Load or simulate an ECG signal of a given duration and sampling rate.
2. Apply a bandpass filter with given cutoff frequencies.
3. Detect R-peaks using neurokit2 or a custom algorithm.
4. Compute instantaneous heart rate and visualize it.
5. Comment on how filtering affects R-peak detection.

Input data:

Duration: 12 s, Sampling Rate: 500 Hz, Filter: 0.5–50 Hz

2. Commands used (or GUI):

```
# Required Libraries
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt
import neurokit2 as nk

# -----
# 1. Simulate ECG Signal
# -----

fs = 500          # Sampling frequency [Hz]
duration = 12     # Signal duration [s]

# Simulated ECG signal
ecg = nk.ecg_simulate(duration=duration, sampling_rate=fs)

# Time axis
time = np.arange(len(ecg)) / fs

# -----
# 2. Bandpass Filtering (0.5-50 Hz)
# -----

def bandpass_filter(signal, lowcut, highcut, fs, order=4):
    nyquist = 0.5 * fs
    low = lowcut / nyquist
    high = highcut / nyquist

    b, a = butter(order, [low, high], btype='band')
    filtered_signal = filtfilt(b, a, signal)

    return filtered_signal

# Apply filter
filtered_ecg = bandpass_filter(ecg, 0.5, 50, fs)

# -----
# 3. R-Peak Detection
# -----

signals, info = nk.ecg_process(filtered_ecg, sampling_rate=fs)

# Extract R-peak locations
r_peaks = info["ECG_R_Peaks"]

# -----
# 4. Heart Rate Calculation
# -----

# RR intervals in seconds
rr_intervals = np.diff(r_peaks) / fs

# Instantaneous heart rate (BPM)
heart_rate = 60 / rr_intervals

# Time positions for HR values
hr_time = r_peaks[1:] / fs

# -----
# 5. Visualization
# -----
```

```
plt.figure(figsize=(14, 10))

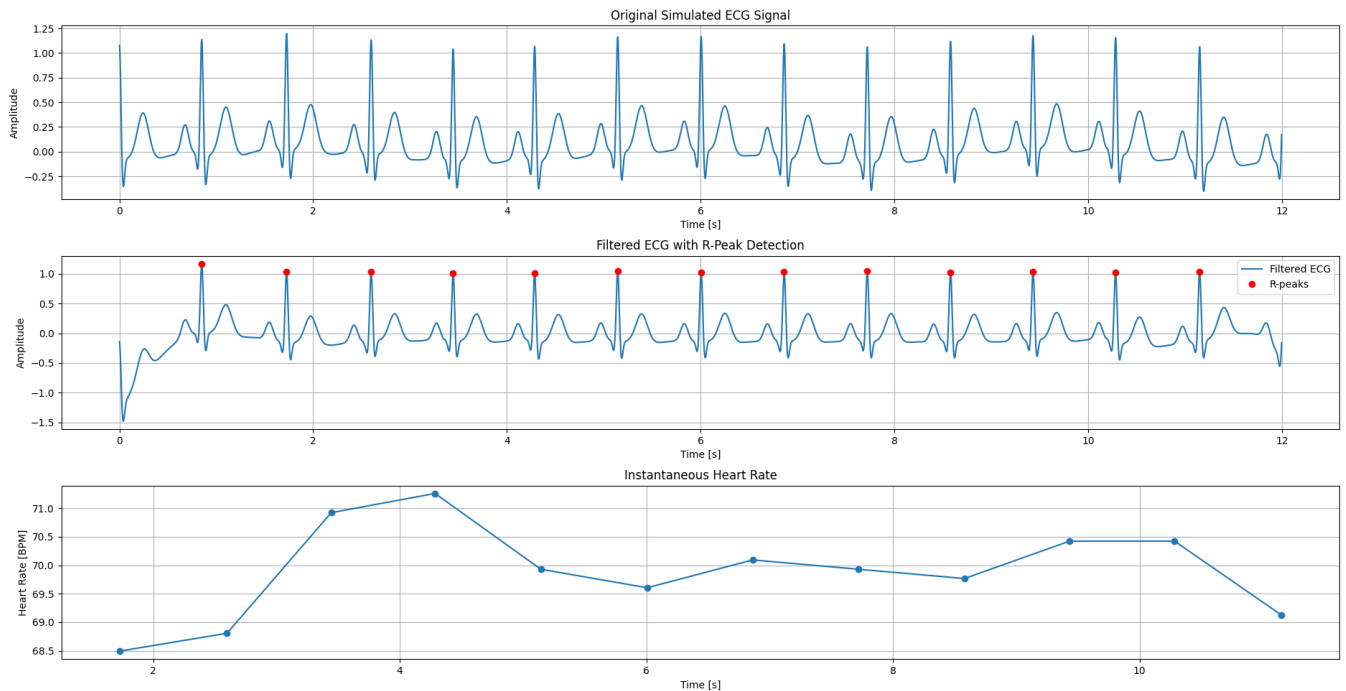
# Original ECG
plt.subplot(3, 1, 1)
plt.plot(time, ecg)
plt.title("Original Simulated ECG Signal")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid(True)

# Filtered ECG with detected R-peaks
plt.subplot(3, 1, 2)
plt.plot(time, filtered_ecg, label="Filtered ECG")
plt.plot(r_peaks / fs, filtered_ecg[r_peaks], 'ro', label="R-peaks")
plt.title("Filtered ECG with R-Peak Detection")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)

# Heart rate over time
plt.subplot(3, 1, 3)
plt.plot(hr_time, heart_rate, marker='o')
plt.title("Instantaneous Heart Rate")
plt.xlabel("Time [s]")
plt.ylabel("Heart Rate [BPM]")
plt.grid(True)

plt.tight_layout()
plt.show()
```

Output:



Conclusion:

Due to this laboratory, we learned that ECG signals can be effectively processed using filtering and peak detection techniques. The application of a 0.5–50 Hz bandpass filter significantly improved signal quality by removing baseline drift and high-frequency noise, which enabled more accurate R-peak detection. Additionally, the analysis of RR intervals allowed for reliable heart rate estimation over time. The experiment demonstrated how proper preprocessing and parameter selection influence the accuracy and stability of ECG analysis results.

Link to remote repository (e.g. GitHub):

<https://github.com/KostiaGrekov/DSPja/tree/main/DSPja%208>